MicroGoal

Backend Development Project

Step 1: Problem Identification & Team Aligment

Many students and professionals struggle to securely manage and track their daily and long-term goals. Existing goal management systems are often overly complex, lack proper authentication mechenisims, or data privacre. There is a need for a simple, secure, and scalable backend system that allows uses to register, authenticate, and manage their goals efficiently. . .

PS ID	User Type	Need	Problem	Root Cause	Impact
PS-1	Student	Manage daily goals online	No simple & secure system	Overcomplicated apps	Frustrated
PS-2	Working Professional	Track and update personal goals	Data privacy concerns	Weak authentication	Insecure
PS-3	Beginner Developer	Build secure goal backend	JWT & middleware complexity	Poor architecture knowledge	Confused
PS-4	General User	Access goals anytime	Systems not properly structured	Unorganized backend design	Unsatisfied

Step 2: Backend Scope Brainstorming

Phase 1
Core Features

User Registration API

- Secure Login using JWT
- Password Hashing using bccypt
- MongoDB Database Integration

General User

Phase 2
Enhancement Features

Goal Categorization

- Deadline Tracking
- Protected Routes M.iddleware

Deadline Tracking

- Update & Delete Goals
- Poor architecture knowledge

Database

- MongoDB Storage & Retrieval

MongoDB

- Express Routes, Modelarrhame, MVC Structure

Step 3: Idea Prioritization

High Priority (Must Have)

- User Authentication (JWT)
- Secure Password Hashing
- Protected Routes
- CRUD Operations

Medium Priority

- Goal Categorization
- Error Handling

Step 3: Idea Prioritization

- **Node Je & Express Server:** Manages all wconing. HTTP. Express and middleware helphing.
- **JWT Authentication:** Token-besed authentication to secure APhenngating.
- **MongoDB Database:** Stores user details and goals efficiently

Why MicroGoal is Unique?

Empathy Map for "MicroGoal" Project

THINK & FEEL

- Is my personal **data safe?**
- Can someone else access my goals?
- Will this system be easy to use?
- Can someone else access my goals?
- **Will** this system be easy to use?
- **Will it** save my **time?**

HEAR

- Close friends suggest productivity tools
- Hears about WhatsApp for reminders
- Backend security is difficult and premium features cost extra

Dharmendar

SEE

- Complex goal apps with subscriptions
- Platforms lacking proper security
- Apps with confusing interfaces

SAY & DO

- I need a **simple system** to manage my goals°
- My data should be **secure**
- Foor authentication systems
- Poor authentication systems

GAINS

- Secure login system
- Privacy goal coverage
- Quick access to personal goals
- Trust in data protection

PAIN POINTS

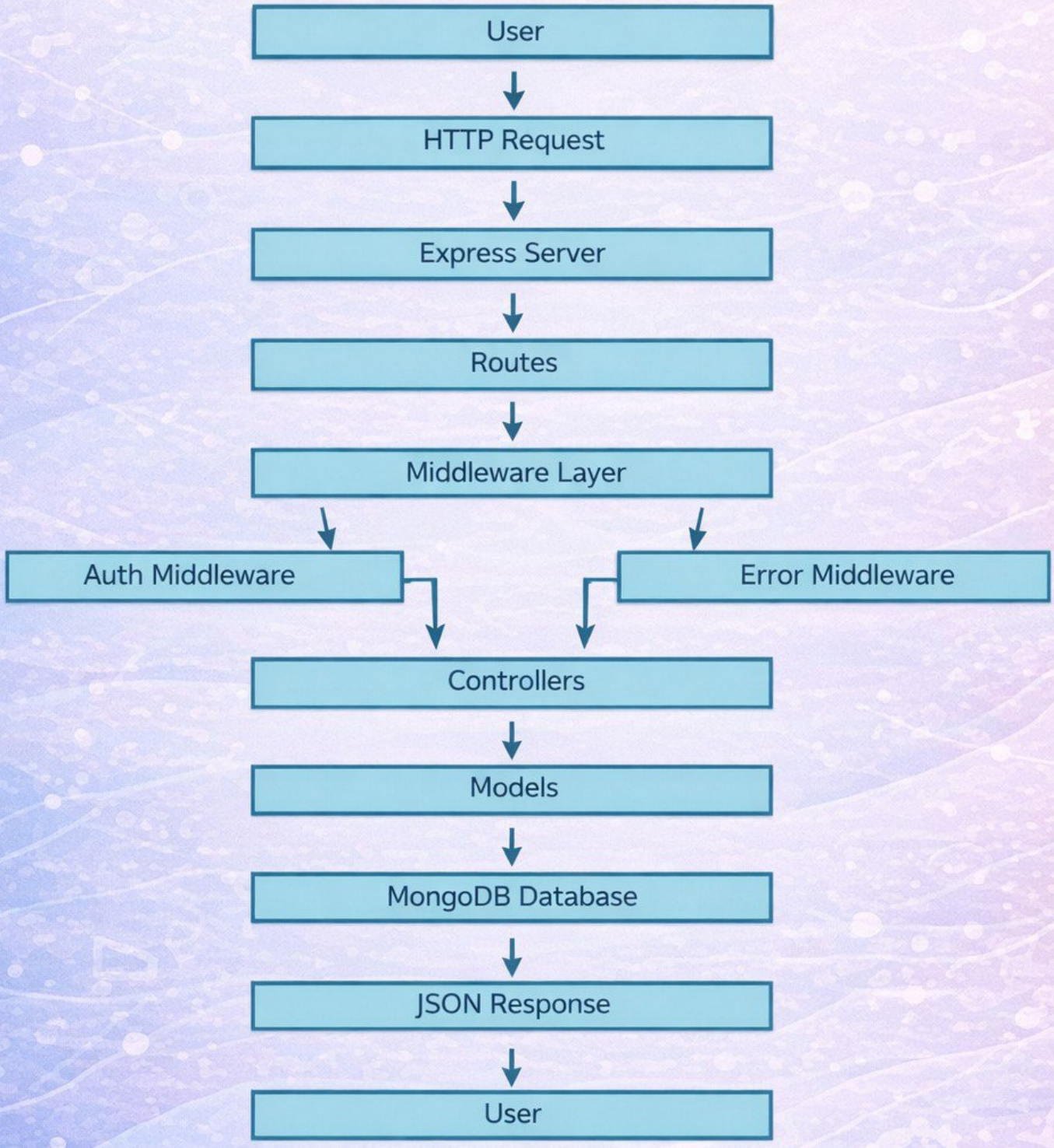
- No secure goal tracking system
- Fear of data leaks
- Time wasted managing tasks manually
- Poor authentication systems
- Complicated interfaces

GAINS

- **Secure** login system
- Privacy goal coverage
- Quick access to personal goals
- Trust in data protection

Express.js HTTP Request Flow with MongoDB

Middleware branches handle auth and errors in parallel



User Stories for MicroGoal Backend API

USN-1 USN-1

As a user, **I want to register** an **account** so that I can access the system.

Acceptance Criteria:

- User data is stored in the database, and a success response is returned.

Priority: **High** | Sprint 1

USN-2 USN-2

As a user, **I want to log in securely** so that I can access my personal goals.

Acceptance Criteria:

- Needs valid email, password, JWT token; returns HTTP status 200.

Priority: **High** | Sprint 1

USN-3 USN-3

As a user, **I want to create** new **goals** so that I can manage my tasks.

Acceptance Criteria:

- Goal is stored in the database, with a success response (HTTP 201).

Priority: **High** | Sprint 1

USN-4 USN-4

As a user, **I want to view** my goals so that I can track my progress.

Acceptance Criteria:

- Only authenticated users can access their own goals.

Priority: **High** | Sprint 1

USN-5 USN-5

As a user, **I want to update** my existing goal so that I can modify my tasks.

Acceptance Criteria:

- Updated goal reflects correctly in the database.

Priority: **High** | Sprint 1

USN-6 USN-6

As a user, **I want to delete a goal** so that I can remove completed or unnecessary tasks.

Acceptance Criteria:

- Goal is permanently deleted and a success response is returned.

Priority: **High** | Sprint 1

USN-9 USN-9

As a **developer / amтni,** I want to test all API endpoints **to** ensure they work **correctly.**

Priority: **Medium** | Sprint: 2 | Sprint 2.

User Stories – MicroGoal Backend API

Register Account

A user with an account to access system.

Acceptance

The user data is storee succesfully and a response is returned.

Login Securely

A user logs in to access personal goals.

Acceptance

A JWT token is generated and returned.

Create Goal

A iser rith a goal to manage their tasks.

Acceptance

The goal is stored in the database and a 201 status is returned.

View Goals

User tracks progress by viewing goals.

Acceptance

Only authenticated users can see their goals.

Delete Goal

User removes completed or unnecessary goals.

Security Protection

Users cannot access other users' goals.

Project Design Phase-II

Technology Stack & Architecture

Technical Architecture

S.No	Component	Description	Technology
1	User Interface	API testing interface for sending requests	Thunder Client
2	Application Logic-1	User authentication and account managenent	Node.js + Express.js
3	Application Logic-2	Goal CRUD operations and data processing	Node.js + Express.js
4	Authentication Layer	Token generation and verification	JSON Web Token (JWT)
5	Database Layer	Store users and goals data	MongoDB + Mongoose

Technology Stack

S.No	Characteristics	Description	Technology
	Open-Source Frameworks	Node.js & Express.js backend development framework	Node.js, & Express.js
	Database	NoSQL datasse for storing structured data	MongoDB
	Security	Secure API access using token-based authentication	JWT
	API Communication	RESTful API uscing HTTP (GET, POST, PUT, PUT, DELETE)	Express.js
	Scalable Architecture	MVC Pattem Modular MVC structure for future expansion	MVC Pattern
	API Testing	Thunder Client	Thunder Client

Project Planning Phase

Sprint Backlog & Schedule

Project ID	Team	Project Name	Date
USR-4456	Team A	MicroGoal Backend System	16 16 Feb 2026

Sprint Backlog & Schedule

Sprint	Epic	Story Points	Priority	Priority	Team Members
Sprint 1	User Authentication	USN-1		High	Dharmender
Sprint-1	• Implement User Registration API • Implement User Login with JWT	USN-2		High	Dharmender
Sprint-1	Database Setup	USN-3		High	Dharmender
Sprint 2	• Configure MongoDB & Mongoose Models	USN-3		High	Dharmender
Sprint 2	Create Goal API (POST)	USN-4		High	Dharmender
	• Get User Goals API (GET)	USN-5		High	Dharmender
Sprint 2	Goal Management	USN-6		High	Dharmender
Sprint 3	• Implement Auth Middleware	USN-8		High	Dharmender
	• Error Handling	USN-9		Medium	Dharmender
Sprint 4	Aptimization	USN-10		Medium	Dharmender
Sprint 4	• Add Request Validation & Status Codes	USN-11		Medium	Dharmender
	• Performance	USN-12		Medium	Dharmender
Sprint-5	Optimization	USN-12		Medium	Dharmender
Sprint 4	• Add Request Validation & Status Codes	USN-12		Medium	Dharmender

MicroGoal Backend System

Functional Requirements

FR–1	Requirement	Description
	User Authentication & Account Management	• User Registration • User Login • Password Hashing • JWT Token Generation
FR-2	Goal Management (CRUD Operations)	• Create, View, Update, & Delete Goals
FR-3	Authorization & Access Control	• Protect Routes using Auth Middleware

Non-Functional Requirements

NFR-1	Requirement	Description
	Usability	Simple & clear API endpoints for easy developer understanding
	Security	Secure API with JWT authentication & password hashing
	Reliability	Proper error handling, preventing crashes
	Performance	Fast API responses, under 2 seconds

Project Overview

Technology Stack & Architecture

Project Overview

Project Name: MicroGoal

A Node.js and Express-based backend API that allows users to register, log in securely using JWT authentication, and manage personal goals through CRUD operations. The system ensures secure route protection, centralized error handling, and structured data storage using MongoDB.

Testing Scope

Features and Functionalities to be Tested

- User Registration
- User Login with JWT Authentication
- Create Goal API
- View User-Specific Goals
- Update Goal API
- Auth Middleware (Protected Routes)
- Error Handling Middleware
- API Response Time & Status Codes

- Registering a new user account
- Logging in and receiving a valid JWT token
- Creating a goal successfully
- Viewing only authenticated user goals

Test Cases:

Test Case ID	Test Scenario	Test Steps	Actual Result	Pass/Fail
TC-001	User Registration	1. Send POST request to /register	User created successfully	Pass
TC-002	User Login	2. Send POST request to /login	Token generated	Pass
TC-003	Create Goal	3. Send POST request to /goals	Goal created	Pass

- Updating an existing goal
- Deleting a goal
- Preventing unauthorized access (401 error)
- Handling invalid input or server errors gracefully

Purpose

- To design and implement a **secure and efficient** backend system for managing personal goals.
- To provide users with a **robust RESTful API** for creating, modifying, retrieving, and deleting goals.
- To incorporate **JWT-based authentication** to ensure secure access.
- To enforce **role-based access** and authorization controls.
- To develop a **scalable** and **maintainable** backend architecture for future enhancements.

Problem Statement

In the digital era, many individuals prefer managing their personal or long-term goals through online platforms. However, they face several challenges:

- Lack of adequate **security measures** in existing systems.
- **Overly complex** platforms that are difficult to navigate.
- Concerns over **unauthorized access** and data breaches.
- Difficulty for developers in implementing **secure** *authentication* and **error handling**.

Proposed Solution

A Goal Management Backend API built with **Node.js** and **Express.js** to deliver a secure and efficient goal tracking system, featuring:

- **Secure user registration** and login *processes*.
- **JWT-based authentication** for *safeguarding* API **endpoints**.
- **Full CRUD** operations for *comprehensive* goal management.
- **Centralized error handling** *middleware for consistent responses.*
- **MongoDB** *as a NoSQL database for structured data storage.*
- **Modular architecture** *(Routes → Controllers → Models → Middleware) to ensure maintainability and scalability.*

Notes

NMicroGoal Project

- Develop a secure Node.js & Express backend API
- Implement JWT-based user authentication and token generation
- Set up MongoDB database for storing user and goal data
- Manage goal CRUD operations (Create, Read, Update, Delete)
- Protect API routes with Auth middleware
- Implement centralized error handling with Express Middleware
- Ensure API usability, performance, and reliability
- Handle API testing and validation to meet project standards